

RaceDay

Part 2 - RESTful API Development

Student Number: ST10488271

Module: PROG6212

Marks: 100

This document describes the design and implementation of the RaceDay RESTful API, built with ASP.NET Core Web API, Entity Framework Core (Code-First), and session-based role authentication, exactly as planned in Part 1.

1. Overview

Part 2 builds the API that powers the entire RaceDay system, following the endpoint plan produced in Part 1. The API is the backbone of the system: it handles all data operations, enforces role-based access, and will be fully tested before the MVC front end is built in Part 3.

Roles

- **Organiser** - creates, edits, and deletes events; manages event categories; captures participant results; views all enrolments for events they own.
- **Participant** - registers an account, browses events, enters events by selecting a category, views their own enrolments, and tracks their own result history.

Technology Stack

Layer	Technology
API Framework	ASP.NET Core Web API (.NET 8)
ORM / Data Access	Entity Framework Core 8 - Code-First
Database	SQL Server (LocalDB / Express / Developer)
Authentication	Server-side session state (HttpContext.Session)
Password Storage	PBKDF2-SHA256 with per-user random salt
API Documentation	Swagger / Swashbuckle.AspNetCore
Unit Testing	xUnit + Microsoft.AspNetCore.Mvc.Testing + EF Core InMemory
CI/CD	GitHub Actions

2. Project Architecture

The solution is split into two projects so the API and its tests are cleanly separated:

```
RaceDay.sln
■■■ RaceDay.API/ (the Web API project)
■   ■■■ Controllers/ Auth, Users, Events, Categories, Enrolments, Results
■   ■■■ Models/ EF Core entities matching the Part 1 ERD exactly
■   ■■■ Data/ApplicationDbContext.cs
■   ■■■ DTOs/ request / response shapes
■   ■■■ Security/ RequireAuthAttribute, RequireRoleAttribute, SessionKeys
■   ■■■ Services/PasswordHasher.cs
■   ■■■ Program.cs
■■■ RaceDay.Tests/ (xUnit integration tests)
```

2.1 Database Schema (Code-First)

ApplicationDbContext defines the same six entities as the Part 1 ERD, with identical primary keys, foreign keys, and constraints:

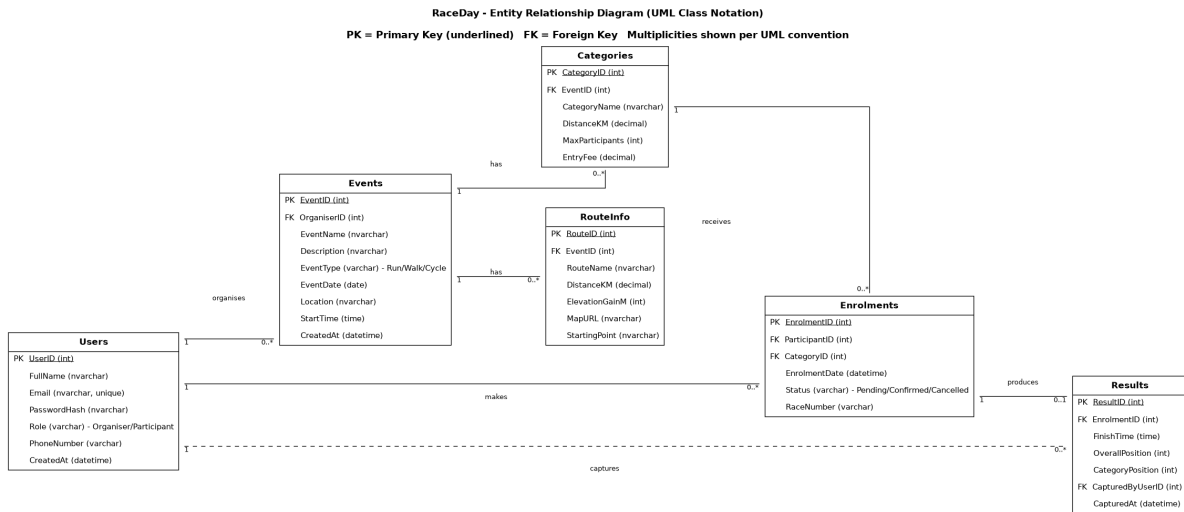


Figure 1: Entity Relationship Diagram (carried over from Part 1, docs/RaceDay_ERD.png). The EF Core Code-First model in ApplicationDbContext.cs reproduces every entity, key, and cardinality shown here - see docs/RaceDay_Database.sql for the equivalent raw SQL.

3. Authentication & Role-Based Access

The brief requires session management to maintain the user's authenticated state and role for all subsequent requests. This API therefore uses ASP.NET Core's built-in server-side session state rather than JWT bearer tokens.

3.1 Registration and Password Storage

POST /api/auth/register accepts full name, email, password, role, and phone number. The role must be exactly 'Organiser' or 'Participant' - anything else returns 400 Bad Request. The password is hashed with PBKDF2-SHA256 (100,000 iterations, 128-bit random salt per user) before it is ever written to the database - the original password is never stored or logged.

3.2 Login and Session Creation

POST /api/auth/login verifies the supplied password against the stored hash. On success, the user's ID and role are written into HttpContext.Session, and ASP.NET Core automatically issues a session cookie to the caller:

```
HttpContext.Session.SetInt32(SessionKeys.UserId, user.UserId);
HttpContext.Session.SetString(SessionKeys.Role, user.Role);
```

3.3 Enforcing Authentication and Roles

Two action filters read the session on every subsequent request to that endpoint:

- **RequireAuthAttribute** - returns 401 Unauthorized if no session exists.
- **RequireRoleAttribute("Organiser") / RequireRoleAttribute("Participant")** - returns 403 Forbidden if the session's role does not match the required role.

Ownership is enforced separately, inside each controller action, by comparing the session's UserId against the resource's owner column (e.g. an Organiser can only update or delete an Event where Event.OrganiserId equals their own session UserId). This is what stops one organiser from editing another organiser's events, and stops any Participant from reaching Organiser-only actions and vice versa.

4. Implemented Endpoints

All endpoints from the approved Part 1 plan were implemented. Two minor, deliberate deviations are explained in docs/Part2_Endpoint_Plan_Deviations.md (the weather endpoint was deferred to Part 3, and a logout endpoint was added). The table below summarises what was built.

Area	Endpoints Implemented	Access
Auth	POST /api/auth/register POST /api/auth/login POST /api/auth/logout	Public / Public / Any
User Profile	GET /api/users/me PUT /api/users/me GET /api/users/{id}	Any / Any / Public

Events	GET /api/events GET /api/events/{id} GET /api/events/{id}/route POST /api/events PUT /api/events/{id} DELETE /api/events/{id}	Public / Public / Public / Organiser / Organiser (owner) / C
Categories	GET /api/events/{eventId}/categories POST /api/events/{eventId}/categories PUT /api/categories/{id} DELETE /api/categories/{id}	Public / Organiser (owner) / Organiser (owner) / Organise
Enrolments	POST /api/enrolments GET /api/enrolments/me GET /api/events/{eventId}/enrolments PUT /api/enrolments/{id}/status DELETE /api/enrolments/{id}	Participant / Participant / Organiser (owner) / Organiser (o
Results	POST /api/results GET /api/results/me GET /api/events/{eventId}/results PUT /api/results/{id}	Organiser (owner) / Participant / Public / Organiser (owne

5. Swagger Integration

Swagger (via Swashbuckle.AspNetCore) is registered in Program.cs and enabled in Development mode, launching automatically at /swagger. Every controller action is annotated with an XML doc-comment summary and typed DTOs for its request body and response, so Swagger UI displays each endpoint's purpose, expected input, and response shape without any extra tooling - acting as live, browsable verification that the implemented API matches the Part 1 plan.

6. Unit Testing

RaceDay.Tests uses xUnit with Microsoft.AspNetCore.Mvc.Testing's WebApplicationFactory to boot the real API pipeline (Program.cs) for each test class, with the SQL Server DbContext swapped out for an isolated EF Core InMemory database (CustomWebApplicationFactory). Because the test HttpClient automatically carries the session cookie between requests, tests can log in once and then call protected endpoints exactly as a real client would - this validates the session mechanism itself, not just the business logic behind it.

Test Class	Scenarios Covered
AuthControllerTests	Register success; duplicate email -> 409; invalid role -> 400; login success creates a working session; v
EventsControllerTests	Public browsing works without login; Organiser can create events; Participant creating an event -> 403;
EnrolmentsControllerTests	Participant can enrol -> 201; Organiser enrolling -> 403; duplicate enrolment -> 409; a participant's own

Run locally with: dotnet test RaceDay.sln. These same tests run automatically in the GitHub Actions workflow on every push.

7. CI/CD Pipeline (GitHub Actions)

.github/workflows/ci-part2.yml runs automatically on every push or pull request to the main branch, and can also be triggered manually from the Actions tab. It performs five steps:

- Checks out the repository code.
- Installs the .NET 8 SDK on the GitHub-hosted runner.
- Restores all NuGet packages for the solution.
- Builds RaceDay.sln in Release configuration.
- Runs every unit test in RaceDay.Tests and uploads the results as a downloadable artifact.

Because the tests use an EF Core InMemory database instead of a real SQL Server connection, the entire pipeline runs without any database server being configured on the runner.

7.1 How to See It Go Green on GitHub

- Push your commits to the main branch of your assigned GitHub repository.
- Open the repository on GitHub.com and click the **Actions** tab.
- You'll see a workflow run named 'Build and Test Part 2 - RaceDay API' - click it to watch it progress.
- A yellow dot means it's running; a green tick means every step (including all unit tests) passed; a red cross means something failed - click the failing step to see the error log.
- Once it's green, take a screenshot of the Actions page showing the green tick and paste it into the README.md where indicated.

8. Setup Instructions

Prerequisites

- Visual Studio 2022 (17.8+) with the 'ASP.NET and web development' workload, or the .NET 8 SDK with any editor.
- SQL Server (LocalDB, Express, or Developer edition) - LocalDB ships with Visual Studio.

Steps

1. Open RaceDay.sln in Visual Studio.
2. Confirm the connection string in RaceDay.API/appsettings.json matches your SQL Server instance.
3. Create the database from the Code-First models:
dotnet ef migrations add InitialCreate
dotnet ef database update
4. Run the API (dotnet run, or F5 in Visual Studio) - Swagger opens automatically.
5. Run the tests: dotnet test RaceDay.sln

This documentation, together with the source code in RaceDay.API and RaceDay.Tests, the ERD and SQL script carried over from Part 1, and the GitHub Actions workflow, forms the complete Part 2 submission for PROG6212.